

SokoPoint — 20-Day Development Workflow

Project: SokoPoint — Multi-vendor E-commerce Platform (Django)
Timeline: 20 working days (4 sprints × 5 days)
Start Date: September 10, 2026
End Date: October 7, 2026
Tech Stack: Django 5.x, Django REST Framework, SQLite → PostgreSQL, HTML/CSS/JS

Sprint Overview

Sprint	Days	Theme	Key Deliverables
Sprint 1	Day 1–5	Foundation & Auth	Project setup, CustomUser model, registration, login, dashboards
Sprint 2	Day 6–10	Product Management	Product model, CRUD, templates, image uploads, admin
Sprint 3	Day 11–15	Cart, Orders & Payments	Shopping cart, checkout flow, payment processing, order management
Sprint 4	Day 16–20	API, Testing & Deployment	REST API, JWT auth, testing, polish, deployment

Sprint 1 — Foundation & Authentication (Days 1–5)

Day 1 — Project Setup & Configuration

Jira Story: SP-001 — *Set up Django project and development environment*

Description:

Initialize the Django project, create the app structure, configure settings, and set up version control.

Sub-tasks:

#	Sub-task	Type
1.1	Create Python virtual environment and install Django 5.x	Task
1.2	Run <code>django-admin startproject SokoPoint</code>	Task
1.3	Create apps: <code>account</code> , <code>product</code> , <code>order</code> , <code>cart</code>	Task
1.4	Register all apps in <code>INSTALLED_APPS</code>	Task
1.5	Configure <code>TEMPLATES.DIRS</code> for each app's template folder	Task
1.6	Configure <code>STATIC_URL</code> , <code>MEDIA_URL</code> , <code>MEDIA_ROOT</code>	Task
1.7	Initialize Git repo, create <code>.gitignore</code> , make initial commit	Task

Acceptance Criteria:

- ☐ `python manage.py runserver` starts without errors
 - ☐ All 4 apps are registered and recognized by Django
 - ☐ Git repo initialized with clean initial commit
 - ☐ Project structure matches the intended architecture
-

 Day 2 — Custom User Model

Jira Story: SP-002 — *Create CustomUser model with vendor/customer roles*

Description:

Build the CustomUser model extending `AbstractBaseUser` + `PermissionsMixin` with a custom manager. Users have a `user_type` field (customer or vendor) plus profile fields (phone, location).

Sub-tasks:

#	Sub-task	Type
2.1	Create CustomUserManager with <code>create_user()</code> and <code>create_superuser()</code>	Task
2.2	Create CustomUser model with fields: <code>username</code> , <code>first_name</code> , <code>last_name</code> , <code>email</code> , <code>phone</code> , <code>location</code> , <code>user_type</code> , <code>is_staff</code> , <code>is_active</code> , <code>date_joined</code>	Task
2.3	Define <code>USER_TYPE_CHOICES</code> : <code>customer</code> , <code>vendor</code>	Task
2.4	Set <code>AUTH_USER_MODEL = 'account.CustomUser'</code> in settings	Task
2.5	Add password validation in the custom manager	Task
2.6	Run <code>makemigrations</code> and <code>migrate</code>	Task
2.7	Create a superuser for testing	Task

Acceptance Criteria:

- ☐ CustomUser is the active auth model
 - ☐ Superuser can be created via `createsuperuser`
 - ☐ Password validation works on user creation
 - ☐ Migration runs cleanly with no errors
-

 Day 3 — User Registration (Customer & Vendor)

Jira Story: SP-003 — *Implement customer and vendor registration flows*

Description:

Build registration forms and views for both customer and vendor user types, with password confirmation and validation.

Sub-tasks:

#	Sub-task	Type
---	----------	------

3.1 Create CustomerRegisterForm (ModelForm) with fields: username, first_name, last_name, email, phone, location, password, confirm_password	Task
3.2 Create VendorRegisterForm with fields: username, email, phone, location, password, confirm_password	Task
3.3 Add password match validation in clean() method	Task
3.4 Add Django password strength validation in clean_password()	Task
3.5 Create customer_register view — sets user_type='customer' on save	Task
3.6 Create vendor_register view — sets user_type='vendor' on save	Task
3.7 Create customer_register.html template	Task
3.8 Create vendor_register.html template	Task
3.9 Add URL routes: /account/register/customer/, /account/register/vendor/	Task

Acceptance Criteria:

- [] Customer can register and is saved with user_type='customer'
- [] Vendor can register and is saved with user_type='vendor'
- [] Mismatched passwords show a validation error
- [] Weak passwords are rejected
- [] After registration, user is redirected to login page

Day 4 — Login, Logout & Base Template

Jira Story: SP-004 — *Implement authentication and create base template*

Description:

Build login/logout functionality with role-based redirects, and create the base HTML template with navigation.

Sub-tasks:

#	Sub-task	Type
4.1	Create user_login view with authenticate() and login()	Task
4.2	Implement role-based redirect: vendors → vendor dashboard, customers → customer dashboard	Task
4.3	Create user_logout view	Task
4.4	Create login.html template with error handling	Task
4.5	Create base.html with full navigation (links to: home, products, cart, login/register, dashboard, logout)	Task
4.6	Add conditional nav items based on user.is_authenticated and user.user_type	Task
4.7	Style base.html with CSS variables, responsive layout, and modern design	Task
4.8	Configure LOGIN_URL and LOGIN_REDIRECT_URL in settings	Task
4.9	Add URL routes: /account/login/, /account/logout/	Task

Acceptance Criteria:

- [] Users can log in with username/password
- [] Invalid credentials show an error message
- [] Vendors are redirected to vendor dashboard after login
- [] Customers are redirected to customer dashboard after login
- [] Logout clears the session and redirects to login
- [] Navigation shows correct links based on auth state

Day 5 — Dashboards & Home Page

Jira Story: SP-005 — *Build vendor dashboard, customer dashboard, and public home page*

Description:

Create role-specific dashboards and a public home page showing available products.

Sub-tasks:

#	Sub-task	Type
5.1	Create <code>vendor_dashboard</code> view — shows vendor's own products, protected with <code>@login_required</code>	Task
5.2	Create <code>customer_dashboard</code> view — shows all available products, protected with <code>@login_required</code>	Task
5.3	Add access control: vendors can't access customer dashboard and vice versa	Task
5.4	Create <code>vendor_dashboard.html</code> template with product listing and management actions	Task
5.5	Create <code>customer_dashboard.html</code> template with product browsing	Task
5.6	Create home view — public page showing latest products (<code>status=True</code>)	Task
5.7	Create <code>home.html</code> template	Task
5.8	Wire up root URL / to home view	Task

Acceptance Criteria:

- [] Vendor dashboard shows only that vendor's products
- [] Customer dashboard shows all active products
- [] Non-authenticated users are redirected to login
- [] Home page is publicly accessible and shows products
- [] Role-based access control is enforced

Sprint 2 — Product Management (Days 6–10)

Day 6 — Product Model & Database

Jira Story: SP-006 — *Create Product model with all fields and choices*

Description:

Define the Product model with currency, unit choices, image upload, and vendor foreign key.

Sub-tasks:

#	Sub-task	Type
6.1	Create Product model with fields: vendor (FK to CustomUser), name, price, currency, description, unit, image, status, created_at, updated_at	Task
6.2	Define CURRENCY_CHOICES: USD, RWF, EUR, KES	Task
6.3	Define UNIT_CHOICES: kg, g, pcs, lit, ml, m, cm, mm, inch, foot, yard, bag, box, crate, bunch	Task
6.4	Set image as ImageField with upload_to='product_images/'	Task
6.5	Set status as BooleanField (True = available)	Task
6.6	Install Pillow for image handling	Task
6.7	Run makemigrations and migrate	Task

Acceptance Criteria:

- ☐ Product model migrates cleanly
- ☐ ForeignKey to CustomUser works with related_name='products'
- ☐ All currency and unit choices render correctly
- ☐ Image upload directory is configured

Day 7 — Add & Edit Product (Vendor)

Jira Story: SP-007 — *Implement product creation and editing for vendors*

Description:

Build forms and views for vendors to add new products and edit existing ones.

Sub-tasks:

#	Sub-task	Type
7.1	Create ProductForm (ModelForm) with fields: name, price, currency, description, unit, image, status	Task
7.2	Create add_product view — only vendors can access, auto-sets vendor=request.user	Task
7.3	Create edit_product view — only the product's vendor can edit	Task
7.4	Create add_product.html template with form styling	Task
7.5	Create product_edit.html template pre-filled with current data	Task
7.6	Add URL routes: /product/add/, /product/<id>/edit/	Task

Acceptance Criteria:

- ☐ Vendor can create a product with image upload
- ☐ Product is automatically linked to the logged-in vendor
- ☐ Only the product's owner can edit it
- ☐ Non-vendors are redirected away
- ☐ Form validation errors are displayed

Day 8 — Product List & Detail Views

Jira Story: SP-008 — Build product listing and detail pages

Description:

Create views and templates for browsing products and viewing individual product details.

Sub-tasks:

#	Sub-task	Type
8.1	Create <code>product_list</code> view — vendors see their own products, customers see all	Task
8.2	Create <code>product_detail</code> view — shows full product info	Task
8.3	Create <code>product_list.html</code> with responsive grid layout and product cards	Task
8.4	Create <code>product_detail.html</code> with image, description, price, and action buttons	Task
8.5	Add "Add to Cart" button for customers, "View Details" for vendors	Task
8.6	Handle products with no image (show placeholder)	Task
8.7	Add URL routes: <code>/product/</code> , <code>/product/<id>/</code>	Task

Acceptance Criteria:

- ☐ Product grid displays with images, names, prices
- ☐ Product cards have hover effects
- ☐ Detail page shows all product information
- ☐ Vendors see "View Details", customers see "Add to Cart"
- ☐ Products without images show a placeholder

Day 9 — Delete Product & Product Admin

Jira Story: SP-009 — Implement product deletion and Django admin registration

Description:

Allow vendors to delete their own products, and register all models in Django admin.

Sub-tasks:

#	Sub-task	Type
9.1	Create <code>delete_product</code> view with POST-only confirmation	Task
9.2	Create <code>product_delete.html</code> confirmation template	Task
9.3	Add ownership check: only product vendor can delete	Task
9.4	Register <code>Product</code> model in admin with <code>list_display</code> , <code>list_filter</code> , <code>search_fields</code>	Task
9.5	Register <code>CustomUser</code> in admin extending <code>UserAdmin</code> with profile fieldset	Task
9.6	Configure <code>admin.site.site_header</code> , <code>site_title</code> , <code>site_url</code>	Task
9.7	Add URL route: <code>/product/<id>/delete/</code>	Task

Acceptance Criteria:

- ☐ Vendor can delete their own products
- ☐ Deletion requires POST confirmation

- ☐ Other users cannot delete products they don't own
- ☐ Admin panel shows Product and CustomUser with proper columns
- ☐ Admin header shows "SOKOPOINT MANAGEMENT"

Day 10 — Sprint 2 Review & Template Polish

Jira Story: SP-010 — *Polish product templates and fix integration bugs*

Description:

Review all product features end-to-end, polish UI, fix edge cases, and ensure smooth vendor workflow.

Sub-tasks:

#	Sub-task	Type
10.1	End-to-end test: vendor registers → logs in → adds product → edits → deletes	Task
10.2	End-to-end test: customer registers → logs in → browses products → views detail	Task
10.3	Fix any broken template links or missing CSS	Bug
10.4	Ensure <code>MEDIA_URL</code> serves uploaded images in development	Task
10.5	Add <code>urlpatterns += static(...)</code> for media files in project URLs	Task
10.6	Verify vendor dashboard correctly links to add/edit/delete actions	Task

Acceptance Criteria:

- ☐ Complete vendor workflow works without errors
 - ☐ Complete customer browsing workflow works without errors
 - ☐ All uploaded images display correctly
 - ☐ No broken links or template errors
-

Sprint 3 — Cart, Orders & Payments (Days 11–15)

Day 11 — Session-based Shopping Cart

Jira Story: SP-011 — *Implement session-based shopping cart*

Description:

Build a Cart class using Django sessions, with add/remove/update functionality and a context processor.

Sub-tasks:

#	Sub-task	Type
11.1	Create <code>Cart</code> class in <code>cart/cart.py</code> with session storage	Task
11.2	Implement <code>add()</code> method with quantity support and override option	Task
11.3	Implement <code>remove()</code> method	Task
11.4	Implement <code>__iter__()</code> to yield cart items with product objects and totals	Task

- | | |
|--|------|
| 11.5 Implement <code>__len__()</code> , <code>get_total_price()</code> , <code>get_total_items()</code> , <code>clear()</code> | Task |
| 11.6 Create <code>cart_context</code> context processor — make cart available in all templates | Task |
| 11.7 Register context processor in <code>TEMPLATES</code> settings | Task |

Acceptance Criteria:

- ☐ Cart persists across page navigations via session
- ☐ Adding same product increases quantity
- ☐ Cart total price calculates correctly
- ☐ Cart item count shows in navigation (via context processor)
- ☐ Cart handles deleted products gracefully

Day 12 — Cart Views & Templates

Jira Story: SP-012 — *Build cart views, templates, and URL routes*

Description:

Create views for adding/removing items and viewing the cart detail page.

Sub-tasks:

#	Sub-task	Type
12.1	Create <code>add_to_cart</code> view (POST only) — adds product and redirects to cart	Task
12.2	Create <code>remove_from_cart</code> view (POST only) — removes product and redirects to cart	Task
12.3	Create <code>cart_detail</code> view — displays all cart items	Task
12.4	Create <code>cart_detail.html</code> with item list, quantities, prices, totals, and checkout button	Task
12.5	Add success messages on add/remove actions	Task
12.6	Add URL routes: <code>/cart/</code> , <code>/cart/add/<id>/</code> , <code>/cart/remove/<id>/</code>	Task
12.7	Update <code>product_list.html</code> — wire "Add to Cart" form to the correct URL	Task

Acceptance Criteria:

- ☐ Adding a product shows success message and updates cart
- ☐ Removing a product shows success message and updates cart
- ☐ Cart detail page lists all items with prices and quantities
- ☐ Cart shows grand total
- ☐ Empty cart shows a helpful message
- ☐ Checkout button links to the checkout page

Day 13 — Order Model & Checkout Flow

Jira Story: SP-013 — *Create Order/OrderItem/Payment models and checkout page*

Description:

Define the order data models and build the checkout form with delivery and payment method selection.

Sub-tasks:

#	Sub-task	Type
13.1	Create Order model with fields: customer, total_amount, status, delivery_address, district, phone_number, created_at, updated_at	Task
13.2	Define STATUS_CHOICES: PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED	Task
13.3	Define DISTRICT_CHOICES (all 30 Rwanda districts)	Task
13.4	Create OrderItem model: order (FK), product (FK), quantity, price	Task
13.5	Create Payment model: order (FK), amount, status, payment_method, transaction_reference, timestamps	Task
13.6	Create CheckoutForm with fields: phone_number, district, delivery_address, payment_method	Task
13.7	Create checkout view — validates cart not empty, creates Order + OrderItems + Payment	Task
13.8	Create checkout.html template	Task
13.9	Run makemigrations and migrate	Task

Acceptance Criteria:

- [] All three models migrate cleanly
- [] Checkout form pre-fills phone and address from user profile
- [] Checkout creates Order, OrderItems (from cart), and Payment records
- [] Empty cart redirects back with warning
- [] After checkout, redirects to payment processing

Day 14 — Payment Processing & Order Confirmation

Jira Story: SP-014 — *Implement payment processing and order confirmation*

Description:

Build payment forms (Card, Mobile Money, Cash on Delivery) and the order confirmation page.

Sub-tasks:

#	Sub-task	Type
14.1	Create CardPaymentForm with fields: card_number, expiry, cvv	Task
14.2	Create MobileMoneyForm with fields: provider (MTN/Airtel/M-Pesa), phone_number	Task
14.3	Create process_payment view — handles COD (instant), Card, and Mobile Money	Task
14.4	Simulate payment success: set payment status to COMPLETED, order to PROCESSING	Task

14.5	Generate <code>transaction_reference</code> (UUID) on successful payment	Task
14.6	Clear cart after successful payment	Task
14.7	Create <code>payment.html</code> template with dynamic form based on payment method	Task
14.8	Create <code>order_confirmation.html</code> template showing order summary	Task
14.9	Create <code>order_confirmation</code> view	Task
14.10	Add URL routes: <code>/order/checkout/</code> , <code>/order/payment/<id>/</code> , <code>/order/confirmation/<id>/</code>	Task

Acceptance Criteria:

- ☐ Cash on Delivery completes immediately without form
- ☐ Card payment form validates and simulates success
- ☐ Mobile Money form validates and simulates success
- ☐ Cart is cleared after successful payment
- ☐ Confirmation page shows order details and status

Day 15 — Order Management (Customer & Vendor)

Jira Story: SP-015 — *Build order history and vendor order management*

Description:

Create views for customers to view their orders and for vendors to see/confirm incoming orders.

Sub-tasks:

#	Sub-task	Type
15.1	Create <code>customer_orders</code> view — lists user's orders sorted by date	Task
15.2	Create <code>vendor_orders</code> view — lists orders containing that vendor's products	Task
15.3	Create <code>confirm_order</code> view (POST) — vendor marks order as SHIPPED	Task
15.4	Create <code>customer_orders.html</code> with order cards and status badges	Task
15.5	Create <code>vendor_orders.html</code> with order items and confirm button	Task
15.6	Create <code>place_order</code> view for single-product direct orders (bypassing cart)	Task
15.7	Create <code>place_order.html</code> and <code>order_success.html</code> templates	Task
15.8	Register <code>Order</code> , <code>OrderItem</code> , <code>Payment</code> in Django admin with inlines	Task
15.9	Add URL routes: <code>/order/customer/</code> , <code>/order/vendor/</code> , <code>/order/vendor/confirm/<id>/</code> , <code>/order/place/<id>/</code>	Task

Acceptance Criteria:

- ☐ Customer can see all their orders with status
 - ☐ Vendor can see orders containing their products
 - ☐ Vendor can confirm/ship an order
 - ☐ Direct single-product ordering works
 - ☐ Admin panel shows orders with inline order items
-

Sprint 4 — API, Testing & Deployment (Days 16–20)

Day 16 — REST API Setup & User Endpoints

Jira Story: SP-016 — *Set up Django REST Framework and user API endpoints*

Description:

Install DRF and SimpleJWT, create serializers and API views for user registration, login, logout, and profile.

Sub-tasks:

#	Sub-task	Type
16.1	Install <code>django-rest-framework</code> and <code>django-rest-framework-simplejwt</code>	Task
16.2	Add <code>rest_framework</code> to <code>INSTALLED_APPS</code>	Task
16.3	Configure <code>REST_FRAMEWORK</code> with JWT authentication in settings	Task
16.4	Create <code>CustomUserSerializer</code> with password validation and <code>create()</code> / <code>update()</code> methods	Task
16.5	Create <code>LoginSerializer</code> with <code>authenticate()</code> validation	Task
16.6	Create <code>api_register</code> view (POST, AllowAny)	Task
16.7	Create <code>api_login</code> view (POST, AllowAny)	Task
16.8	Create <code>api_logout</code> view (POST, IsAuthenticated)	Task
16.9	Create <code>api_profile</code> view (GET/PUT/PATCH, IsAuthenticated) — ensure <code>.save()</code> is called	Task
16.10	Add JWT token obtain/refresh endpoints	Task
16.11	Add API URL routes: <code>/account/api/register/</code> , <code>/account/api/login/</code> , etc.	Task

Acceptance Criteria:

- ☐ API register creates a user and returns user data
- ☐ API login returns user data and sets session
- ☐ JWT token endpoint returns access/refresh tokens
- ☐ Profile GET returns user data, PUT/PATCH updates and persists
- ☐ Unauthenticated requests to protected endpoints return 401

Day 17 — API Testing & Documentation

Jira Story: SP-017 — *Test all API endpoints and add documentation*

Description:

Write API tests and create documentation for all endpoints.

Sub-tasks:

#	Sub-task	Type
---	----------	------

Polish all templates, ensure responsive design on mobile, fix any remaining bugs.

Sub-tasks:

#	Sub-task	Type
19.1	Audit all templates for consistent styling and CSS variable usage	Task
19.2	Add mobile responsive breakpoints to all pages	Task
19.3	Add hover effects and micro-animations to product cards	Task
19.4	Polish form styling across all pages (inputs, buttons, error messages)	Task
19.5	Add success/error message display component (Django messages framework)	Task
19.6	Verify all template links and navigation are correct	Task
19.7	Fix any remaining bugs from end-to-end testing	Bug
19.8	Add empty state designs (no products, no orders, empty cart)	Task

Acceptance Criteria:

- ☐ All pages look good on desktop and mobile
- ☐ Consistent design language across all pages
- ☐ All Django messages display properly
- ☐ No broken links or template errors
- ☐ Empty states show helpful messages



Day 20 — Deployment Preparation & Final Review

Jira Story: SP-020 — *Prepare for deployment and final project review*

Description:

Configure production settings, prepare deployment, run final checks, and document the project.

Sub-tasks:

#	Sub-task	Type
20.1	Move <code>SECRET_KEY</code> to environment variable	Task
20.2	Set <code>DEBUG = False</code> for production, configure <code>ALLOWED_HOSTS</code>	Task
20.3	Configure PostgreSQL database (or keep SQLite for demo)	Task
20.4	Run <code>python manage.py collectstatic</code>	Task
20.5	Run <code>python manage.py check --deploy</code> and fix all warnings	Task
20.6	Create <code>README.md</code> with project description, setup instructions, and features	Task
20.7	Final end-to-end test of all features	Task
20.8	Tag final release in Git: <code>v1.0.0</code>	Task
20.9	Deploy to hosting platform (e.g., Railway, Render, or PythonAnywhere)	Task

Acceptance Criteria:

- [] `python manage.py check --deploy` passes
 - [] Secret key is not hardcoded
 - [] Static files are collected
 - [] README documents all features and setup steps
 - [] Application is accessible via public URL
 - [] All features work in production
-

Jira Board Setup Guide

Epic

Create one Epic: "SokoPoint E-commerce Platform v1.0"

Stories

Create 20 stories (SP-001 through SP-020), one per day, all linked to the Epic.

Labels

Use these labels for filtering:

Label	Usage
setup	Day 1
auth	Days 2–5
product	Days 6–10
cart	Days 11–12
order	Days 13–15
api	Days 16–17
testing	Days 17–18
ui-polish	Day 19
deployment	Day 20

Story Points (suggested)

- Days 1, 10, 19: **3 points** (setup/polish)
- Days 2, 6, 9, 17, 20: **5 points** (moderate)
- Days 3, 4, 5, 7, 8, 11, 12, 15, 16: **8 points** (complex)
- Days 13, 14, 18: **13 points** (most complex)

Sprint Configuration

Sprint	Duration	Stories
Sprint 1	Sep 10–16	SP-001 → SP-005
Sprint 2	Sep 17–23	SP-006 → SP-010

Sprint 3 Sep 24–30 SP-011 → SP-015

Sprint 4 Oct 1–7 SP-016 → SP-020